

BOP V2 — IMPLEMENTATION PLAN — SY051

Operations Cockpit + Strategy Cockpit

OP001 Tasks · OP002 Goals & OKRs · Fresh BOP tables · DesCampers as UI reference only

v2.0 · 2026-08-05

14 new tables

~15 PBAC perms

Design decision: All tables are built new from scratch. The existing DesCampers `tasks`, `objectives`, and `key_results` tables are **not reused, altered, or migrated**. DC screens continue to run independently on their own tables. The DC codebase serves as a UI pattern reference only.

1. Current State — All Systems

BOP CONSOLE OP001 Tasks STUB · 25 LINES Static table in <code>screens.tsx</code> with mock data. No API, no DB table, no CRUD.	BOP CONSOLE OP002 Goals & OKRs PLACEHOLDER · 14 LINES "Interface coming soon." text. No component, no API, no data model.	DESCAMPERS ADMIN DC-TASKS Task Manager REFERENCE · 721 LINES Working CRUD, periodic recurrence, record linking, reminders, status tabs, search, tags, KR linking. UI patterns reusable.	DESCAMPERS ADMIN DC-GOALS Goals & Objectives REFERENCE · 792 LINES Objectives + Key Results hierarchy. Progress tracking, dashboard widget. UI patterns reusable.
---	--	--	--

2. DesCampers Admin — Reference Evaluation

Role: The DesCampers admin panel is evaluated as a **UI pattern reference** only. Its database tables, Server Actions, and data are completely separate from the BOP system. The BOP console builds fresh tables designed to the full cockpit plan spec from day one — no schema compromises from inheriting DC's simpler structure.

What to learn from DC: Status tab bar with counts, side-panel sticky forms, expandable rows with nested items, reminders banner with urgency bands, click-to-cycle status icons, tag chips, progress bars with color thresholds, periodic recurrence UI with smart date display.

What DC lacks (BOP builds fresh): Dependencies, audit trail, comments, attachments, checklist, SLA engine, automation triggers, templates, kanban/calendar, entity scoping, metrics catalog, nightly compute, exec dashboard, PBAC, multi-tenant, i18n.

DC

DesCampers Features — Reusable UI Patterns Only

DC FEATURE	LINES	BOP REUSE	NOTES
Status tabs with count badges	~40	UI pattern	Adapt for 7-status task machine + 4-status goals
Sticky side-panel form	~130	UI pattern	BOP will use drawers + full pages instead for complex forms
Reminders banner (overdue/today/tomorrow)	~50	UI pattern	Evolve into 8-card stat strip with SLA urgency bands
Click-to-cycle status icon	~15	UI pattern	Adapt for dependency-aware transitions
Expandable rows with nested KRs	~80	UI pattern	Extend for 3-level Goal → Objective → KR tree
MiniProgress color-coded bar	~8	UI pattern	Reuse for KR + goal + objective roll-up display
Periodic recurrence UI	~60	UI pattern	Reference for recurrence form (BOP builds RRULE in P3)
Record linking form (type + id + label)	~20	UI pattern	Adapt for BOP's wider polymorphic ref_object scope
Goals dashboard widget	69	UI pattern	Starting point for exec dashboard layout concepts

3. Gap Analysis — OP001 + OP002 vs. Implementation Plan

OP001 Operations Cockpit — Full Build Scope			
COMPONENT	PLAN SPEC	CURRENT STATE	WORK
op_tasks	30+ columns: polymorphic refs, SLA, checklist JSONB, recurrence, assignee, parent_task_id, department_id, priority 1-4, 7 statuses	Does not exist	Full build
op_task_dependencies	Blocker graph, CHECK no self-ref, cycle detection in RPC	Does not exist	Full build
op_task_events	12 event types, actor_id nullable (system), payload JSONB, audit trail	Does not exist	Full build
op_task_comments	Threaded, soft-delete, @mention support	Does not exist	Full build
op_task_attachments	Supabase Storage: op-tasks/{id}/{uuid}.{ext}	Does not exist	Full build
op_task_templates	Reusable chains (is_chain), due/SLA offsets per step	Does not exist	Phase 2
op_task_template_steps	Ordered steps, depends_on_prev, assignee rules	Does not exist	Phase 2
op_task_triggers	Event → template mapping, idempotency constraint, condition SQL	Does not exist	Phase 2

COMPONENT	PLAN SPEC	CURRENT STATE	WORK
<code>op_task_views</code>	Saved filter sets / smart queues, pinnable	Does not exist	Phase 2
Status state machine	7 statuses: open, in_progress, waiting_dependency, waiting_input, waiting_approval, blocked, done. Dependency-aware transitions.	None	Full build
RPCs	<code>task_create</code> , <code>task_transition</code> , <code>task_assign</code> , <code>task_bulk_update</code> , <code>task_snooze</code> — all write events transactionally	None	Full build
API routes	CRUD + cockpit-summary + filtered list at <code>/api/bop/ops/tasks</code>	No <code>/api/bop/ops/directory</code>	Full build
Cockpit UI	8-card stat strip, banded urgency work queue, Grid/Kanban/Calendar views	Static 25-line table stub	Full build
Task detail	Drawer + full page: header, status stepper, checklist, deps, timer, comments, attachments, activity timeline	None	Full build
SLA engine	10-min PM2 cron, 3 escalation levels, Telegram alerts	None	Full build
Automation engine	<code>fireTaskTriggers()</code> wired into inquiry/listing/invoice/support flows	None	Phase 2
Analytics tab	Completed/week, SLA compliance %, workload by dept, auto vs manual ratio	None	Phase 3
PBAC	8 permissions: <code>ops.task.read create update assign close delete</code> , <code>ops.template.manage</code> , <code>ops.trigger.manage</code>	None	Full build

OP002

Strategy Cockpit — Full Build Scope

COMPONENT	PLAN SPEC	CURRENT STATE	WORK
<code>op_goals</code>	Hierarchical (<code>parent_goal_id</code>), entity-scoped (5 entities), period-bound (year/quarter/month), health (<code>on_track/at_risk/off_track</code>), <code>vision_text</code>	Does not exist	Full build
<code>op_goal_key_results</code>	<code>metric_type</code> (count/sum/avg/ratio/manual), direction, baseline/target/current, weight for roll-up, <code>metric_source</code> FK	Does not exist	Full build
<code>op_goal_kr_snapshots</code>	Daily trend history: <code>kr_id</code> , date, value, <code>progress_pct</code> , <code>pace_delta</code> — for sparklines	Does not exist	Full build
<code>op_goal_metrics_catalog</code>	Registered metric codes, whitelisted queries, entity-scoped flag, name i18n JSONB	Does not exist	Phase 2
<code>op_goal_checkins</code>	Confidence 1-5, progress note, blockers, <code>goal_id</code> FK	Does not exist	Full build

Component	Plan Spec	Current State	Work
Goal hierarchy	Goal → Objective → KR (3-level via parent_goal_id self-ref)	None	Full build
Entity scoping	ENUM: des_mobil, des_campers, des_shop, des_systems, des_group	None	Full build
Health system	Auto from pace: >=-5 on_track, -15...-5 at_risk, <-15 off_track. Manual override.	None	Full build
Bridge FK	op_tasks.goal_kr_id → op_goal_key_results	None	Phase 2
Metric service	computeMetric(code, {period, entity}) reading listings, invoices, inquiries, reviews	None	Phase 2
Nightly job	PM2 cron: auto-compute KRs, insert snapshots, health transitions, Telegram on at_risk/off_track	None	Phase 2
Exec dashboard	Vision banner (editable), progress rings, entity lanes, at-risk panel, trend charts from snapshots	14-line placeholder	Full build
Goal tree	3-level expandable: Goal → Objective → KR, inline progress, health dots, roll-up %	None	Full build
Goal detail page	KR list with sparklines, linked tasks, check-in timeline, budget card, close-out flow	None	Full build
PBAC	7 permissions: strategy.goal.read create update close, strategy.kr.update, strategy.checkin.create, strategy.metric.manage	None	Full build

4. Build Phases & Task Breakdown



OPS-P1 Operations Cockpit — Core Engine			~12-15 sessions · Priority: P1	
P	Task	Type	Deliverables	Est
I	T1.1 Create op_tasks table + indexes 30+ columns per plan A2.1. tenant_id UUID NOT NULL FK. Polymorphic ref_object_type/ref_object_id. Priority 1-4. 7 statuses (ENUM). assignee_id FK → bop_users. department_id. sla_due_at TIMESTAMPTZ. escalation_level (0-3). checklist JSONB. parent_task_id self-ref FK. Soft-delete (deleted_at). Recurrence fields: is_recurring, rrule TEXT. created_by FK. 4 indexes: (tenant_id, status, sla_due_at), (tenant_id, assignee_id, status), (ref_object_type, ref_object_id), (tenant_id). RLS: tenant_id = current_setting('app.tenant_id').	schema	DB_TABLE, MIGRATION, RLS_POLICY, 4 indexes	4h
I	T1.2 Create op_task_dependencies task_id FK, depends_on_task_id FK. CHECK: task_id != depends_on_task_id.	schema	DB_TABLE, MIGRATION	1h

P	TASK	TYPE	DELIVERABLES	EST
	Unique(task_id, depends_on_task_id). Cycle detection enforced in RPC layer, not DB constraint.			
I	T1.3 Create op_task_events 12 event types: created, status_changed, assigned, priority_changed, sla_warning, sla_breached, escalated, commented, attachment_added, checklist_toggled, snoozed, closed. actor_id nullable (system events). payload JSONB. Index on (task_id, created_at DESC).	schema	DB_TABLE, MIGRATION	1h
I	T1.4 Create op_task_comments + op_task_attachments Comments: task_id FK, author_id FK, body TEXT, parent_comment_id (threading), deleted_at (soft-delete). Attachments: task_id FK, uploader_id FK, file_name, file_size, mime_type, storage_path TEXT (pattern: op-tasks/{task_id}/{uuid}.{ext}), deleted_at.	schema	2x DB_TABLE, 2x MIGRATION	1h
I	T1.5 Status state machine + RPCs 7 statuses with adjacency matrix. Blocked tasks cannot leave blocked until all blockers resolved. Waiting states require explicit unblock. All RPCs write op_task_events transactionally. task_create: validate + insert + event. task_transition: check adjacency + deps + write event. task_assign: set assignee + event. task_bulk_update: batch status/assignee/priority. task_snooze: set snooze_until + event.	api	5x RPC, adjacency matrix	8h
I	T1.6 API route handlers /api/bop/ops/tasks GET /tasks: filterable by status/priority/assignee/dept/ref_object, server-side pagination, sort by sla_due_at/created_at/priority. POST /tasks: Zod-validated create. PATCH /tasks/[id]: update/transition. DELETE /tasks/[id]: soft-delete. GET /tasks/cockpit-summary: dashboard counters (open, due today, overdue, waiting x3, completed today, SLA at risk).	api	API_ROUTE x2, Zod schemas	6h
I	T1.7 OP001 Cockpit main screen — full build Replace stub. 8-card stat strip (Open, Due Today, Overdue, Waiting Input, Waiting Approval, Waiting Dep, Completed Today, SLA At Risk). Banded urgency work queue: Overdue (red) / Today (orange) / This Week (default) / Later (dim) / Waiting (blue). Quick actions per row: status cycle, assign, snooze. Status tab bar with counts.	screen	SCREEN, I18N_KEY x5	12h
I	T1.8 Task detail drawer + full page Header: task number, status stepper (7-dot), priority badge, type, dept, assignee avatar. Sections: relation card (linked object), checklist (inline toggle with progress), dependency list (blockers), comments (threaded), attachments (upload/preview/download), activity timeline (from op_task_events).	screen	SCREEN x2 (drawer + page), I18N_KEY	10h
I	T1.9 Grid view with server pagination DataGrid with sortable/filterable columns. Server-side pagination. Multi-select checkbox column. Bulk action toolbar appears on selection: reassign, reschedule, close, change priority.	screen	SCREEN	6h
I	T1.10 Quick-add modal Global "+ Task" button in shell bar. Pre-fills ref_object_type/id from current route context via route-to-object resolver map. Minimal form: title, priority, due, assign.	screen	SCREEN	3h
I	T1.11 PBAC permissions + object registry 8 permissions registered. op_tasks as L2 in bop_objects + object-manifest.ts. RLS policies using check_pbac() for own vs all scope.	infra	8x PERMISSION,	2h

P	TASK	TYPE	DELIVERABLES	EST
			bop_objects rows	
I	T1.12 SLA & escalation cron (10-min) PM2 cron worker. Query: tasks WHERE sla_due_at IS NOT NULL AND status NOT IN (done, cancelled). L0 → L1 at SLA-2h: Telegram warning, set escalation_level=1. L1 → L2 at SLA: Telegram alert, pin in cockpit. L2 → L3 at SLA+4h: Telegram escalation to admin. Each transition writes op_task_events.	infra	CRON worker, TELEGRAM x3	4h
I	T1.13 Telegram notifications DesAssistant22bot. Triggers: P1 task created, task overdue, SLA breach. Format: task number, title, assignee, deep link to /console/ops/tasks/[id]. Phase 1 = text only, no inline buttons.	infra	TELEGRAM x2	2h

STR-P1 Strategy Cockpit — Skeleton with Manual KRs			~7-9 sessions · Priority: P1	
P	TASK	TYPE	DELIVERABLES	EST
I	T2.1 Create op_goals table tenant_id UUID NOT NULL FK. Hierarchical: parent_goal_id self-ref FK (NULL = top-level goal, non-NULL = objective). Entity-scoped: entity ENUM (des_mobil, des_campers, des_shop, des_systems, des_group). Period: period_type ENUM (year/quarter/month), period_start DATE, period_end DATE. Status: active/achieved/semi_achieved/paused/closed. Health: on_track/at_risk/off_track + health_override. owner_id FK → bop_users. vision_text TEXT. budget_amount NUMERIC, budget_currency. Soft-delete. Index on (tenant_id, entity, status). RLS: tenant_id = current_setting('app.tenant_id').	schema	DB_TABLE, MIGRATION, RLS_POLICY	3h
I	T2.2 Create op_goal_key_results tenant_id UUID NOT NULL FK. goal_id FK. metric_type ENUM: count, sum, avg, ratio, manual. direction ENUM: increase, decrease. baseline NUMERIC, target NUMERIC, current_value NUMERIC. weight NUMERIC DEFAULT 1 (for roll-up). metric_source VARCHAR (FK to metrics catalog, nullable in P1). unit_label TEXT (e.g. "vehicles", "EUR", "%"). Status same as goals. Soft-delete. tenant_id needed for dashboard aggregate queries + nightly cron batch reads.	schema	DB_TABLE, MIGRATION	2h
I	T2.3 Create op_goal_kr_snapshots + op_goal_checkins Snapshots: kr_id FK, snapshot_date DATE, value NUMERIC, progress_pct SMALLINT, pace_delta NUMERIC. UNIQUE(kr_id, snapshot_date). Check-ins: goal_id FK, author_id FK, confidence SMALLINT (1-5), progress_note TEXT, blockers TEXT, created_at.	schema	2x DB_TABLE, 2x MIGRATION	2h
I	T2.4 Goal/Objective CRUD API routes /api/bop/ops/goals : GET (tree query with recursive CTE for hierarchy, KRs nested, progress computed), POST (create goal or objective under parent), PATCH (update, close-out flow), DELETE (soft). Manual KR value entry: PATCH /goals/kr/[id] with current_value. GET /goals/dashboard-summary for exec dashboard counters.	api	API_ROUTE x2, Zod schemas	6h
I	T2.5 OP002 Goal tree screen — full build Replace placeholder. 3-level expandable tree: Goal → Objectives → KRs. Each node shows: title, entity badge, period badge, health dot (green/amber/red), progress bar with weighted roll-up %. Row actions: check-in, edit, add child, close-out. "+ Goal" and scoped "+ Objective" buttons. Status filter tabs. Search.	screen	SCREEN, I18N_KEY x5	12h

P	TASK	TYPE	DELIVERABLES	EST
I	T2.6 Goal/Objective detail page Header: number, title, owner, entity badge, period badge, status, health dot. KR list with manual value editor (slider/input), progress bars, unit labels. Check-in form: confidence slider (1-5), blockers textarea, progress note. Check-in history timeline. Budget mini-card (amount vs spent). Close-out form with final assessment.	screen	SCREEN, I18N_KEY x5	10h
I	T2.7 Weighted roll-up computation On-write RPC (not DB trigger). KR progress = (current - baseline) / (target - baseline) * 100, clamped 0-100, direction-aware. Goal progress = weighted avg of child KR progress. Parent goal = weighted avg of child goal progress. Health auto from pace: compare expected linear % at this date vs actual %. Fire on every KR value update or goal status change.	api	RPC	3h
I	T2.8 PBAC permissions + object registry 7 permissions. Register op_goals (L2) and op_goal_metrics_catalog (L3) in bop_objects.	infra	7x PERMISSION, bop_objects rows	1h
I	T2.9 Seed Q3-2026 goals data Seed initial goals matching current business focus. E.g., Goal: "Grow DES Campers trading rhythm" → Objectives: "Increase caravan sales volume", "Improve inquiry conversion" → KRs with manual values for now.	schema	MIGRATION (seed SQL)	1h

OPS-P2 Operations Cockpit — Automation & Views			~8-10 sessions · Priority: P2	
P	TASK	TYPE	DELIVERABLES	EST
I	T3.1 Create op_task_templates + op_task_template_steps Templates: name i18n JSONB, is_chain BOOL, default_priority, category. Steps: template_id FK, step_order, title, due_offset_hours, sla_offset_hours, assignee_rule TEXT, depends_on_prev BOOL.	schema	2x DB_TABLE	2h
I	T3.2 Create op_task_triggers + op_task_trigger_events Triggers: event_type TEXT, template_id FK, condition_sql TEXT (optional predicate), enabled BOOL. Events (log): trigger_id FK, source_object_type, source_object_id, tasks_created INT. UNIQUE(trigger_id, source_object_type, source_object_id) for idempotency.	schema	2x DB_TABLE	2h
I	T3.3 fireTaskTriggers() service TypeScript service. Single entry point for all auto-generated tasks. Matches event_type → active triggers → evaluates condition_sql → instantiates template steps as op_tasks. Idempotency check before insert. Returns created task IDs for logging.	api	SERVICE	8h
I	T3.4 Wire triggers into existing BOP flows 5 integration points: listing_inquiry.created → inquiry_response template. listing.status='purchased' → vehicle_intake_chain (5 chained steps). fin_invoice.overdue → invoice_reminder. sup_request.created → support_first_response. review.score<3 → review_followup.	integration	5x integration points	6h

P	TASK	TYPE	DELIVERABLES	EST
I	T3.5 Seed 5 automation templates inquiry_response (1 step, 2h SLA), vehicle_intake_chain (5 steps, chained), invoice_reminder (1 step, 48h), support_first_response (1 step, 4h SLA), review_followup (1 step, 24h).	schema	MIGRATION (seed)	2h
I	T3.6 Create <code>op_task_views</code> + saved filters UI Serialized filter model (status, priority, assignee, dept, date range, custom SQL). Pin/unpin. Left rail in cockpit with view list.	screen	DB_TABLE, SCREEN	4h
I	T3.7 Kanban view Columns: Open / In Progress / Waiting (3 lanes collapsed) / Blocked / Done- today. Drag-and-drop = task_transition RPC call. Card shows: title, priority stripe, assignee avatar, due badge.	screen	SCREEN	8h
I	T3.8 Calendar view Tasks positioned by due_at. Day/week/month toggle. Click task to open detail drawer. Color by priority.	screen	SCREEN	6h
I	T3.9 Bulk actions toolbar Appears when multi-select active in grid. Actions: reassign, reschedule (date picker), close, change priority. Each action fires individual task_transition/task_assign RPCs, one event per task.	screen	SCREEN component	3h

STR-P2 Strategy Cockpit — Automated Metrics & Executive Dashboard			~6-8 sessions · Priority: P2	
P	TASK	TYPE	DELIVERABLES	EST
I	T4.1 Create <code>op_goal_metrics_catalog</code> code VARCHAR PK, name_i18n JSONB (NL/EN/DE/FR/TR), metric_type ENUM, unit_label, entity_scoped BOOL, query_key TEXT (internal identifier for computeMetric). 9 initial codes: vans_sold, revenue_invoiced, gross_margin_sum, avg_days_to_sale, inquiries_count, inquiry_to_sale_conversion, nps_avg, review_score_avg, stock_count.	schema	DB_TABLE, seed rows	2h
I	T4.2 <code>computeMetric()</code> service TypeScript service. Switch on query_key. Reads from: listings (sold count via status, days-to-sale from listed → sold delta, stock via active count), fin_invoices (revenue sum, margin sum — margeregeling-aware), listing_inquiries (lead volume, conversion = inquiries with listing.status=purchased / total), reviews (avg score, NPS = promoters–detractors). All entity-scoped and period-filtered.	api	SERVICE	10h
I	T4.3 Nightly computation + snapshots cron PM2 cron worker, daily at 02:00. For each active auto-KR (metric_source IS NOT NULL): computeMetric → update current_value → INSERT op_goal_kr_snapshots. Recompute roll-up for parent goals. Health recalculation. Telegram alert on health transitions (on_track → at_risk, at_risk → off_track).	infra	CRON worker, TELEGRAM	4h
I	T4.4 <code>op_tasks.goal_kr_id</code> FK bridge ADD COLUMN goal_kr_id UUID REFERENCES op_goal_key_results(id) ON DELETE SET NULL. Task detail shows linked KR chip. KR detail page lists open/done linked tasks with counts. Task quick-add from KR detail pre-fills the FK.	schema	MIGRATION, SCREEN updates x2	2h

P	TASK	TYPE	DELIVERABLES	EST
I	T4.5 Executive dashboard Vision banner: editable text field (stored on top-level goal). Top strip: active-quarter KRs as progress rings (Recharts). Entity lanes: DES Mobil / Campers / Shop / Systems rows, each showing their goals + progress. At-risk panel: goals with health=at_risk or off_track, sorted by severity. Trend charts: click KR → line chart from snapshots (last 90 days).	screen	SCREEN, I18N_KEY	12h
I	T4.6 Monday check-in Telegram prompt PM2 cron: Monday 08:00. Message: active objectives + current progress % + pace indicator (arrow up/down/flat). Deep link to /console/ops/goals?action=checkin. Solo-founder ritual.	infra	TELEGRAM, CRON	2h
I	T4.7 KR sparklines + metric-source chips Last 30 snapshots as inline sparkline per KR row. Auto (lightning icon) vs Manual (pen icon) chip. Pace arrow: up/down/flat based on last 7-day trend.	screen	SCREEN updates	4h

OPS-P3 + STR-P3		Advanced — Deferred until go-signals		Go-signal: 8-10 vehicles/month or first staff hire	
P	TASK		PHASE	EST	
I	T5.1 Dependencies UI + Timeline/Gantt view for chained tasks		OPS-P3	8h	
I	T5.2 Recurrence engine (RFC 5545 RRULE parsing + next-occurrence spawning)		OPS-P3	4h	
I	T5.3 Time tracking: start/stop timer + estimate vs actual		OPS-P3	4h	
I	T5.4 Telegram inline actions (Done / Snooze 1d callback buttons)		OPS-P3	3h	
I	T5.5 Analytics tab: completed/week chart, SLA compliance %, workload by dept, auto vs manual ratio		OPS-P3	8h	
I	T6.1 Period close-out flow + historical performance ledger		STR-P3	4h	
I	T6.2 Budget linkage to boekhouding cost categories		STR-P3	4h	
I	T6.3 Forecast projection lines on trend charts (linear pace)		STR-P3	3h	
I	T6.4 Entity scorecards (per-entity fixed metric sets)		STR-P3	6h	

5. Complete Table Schema Overview

14	New BOP Tables — Tenant Scoping Analysis
Rule: <code>tenant_id</code> goes on tables that are directly queried in list views, dashboards, RLS policies, or cross-parent aggregations. Pure child tables accessed only through a parent UUID FK inherit tenant isolation from the parent join — no redundant <code>tenant_id</code>.	

TABLE	MODULE	PHASE	TENANT_ID	KEY COLUMNS	REASONING
op_tasks	OP001	P1	YES	30+ cols: tenant_id, title, status (7-ENUM), priority (1-4), assignee_id, department_id, sla_due_at, escalation_level, checklist JSONB, ref_object_type/id, parent_task_id, is_recurring, rrule, goal_kr_id (P2), deleted_at	RLS root. Cockpit queries, filtered lists, dashboard counters — all scoped by tenant.
op_task_dependencies	OP001	P1	NO	task_id FK, depends_on_task_id FK, CHECK + UNIQUE	Always accessed via task_id. Never listed independently. Tenant flows through op_tasks join.
op_task_events	OP001	P1	NO	task_id FK, event_type (12 types), actor_id, payload JSONB	Activity timeline for one task. Never cross-task query. Tenant via task_id → op_tasks.
op_task_comments	OP001	P1	NO	task_id FK, author_id, body, parent_comment_id, deleted_at	Comments on a specific task. Accessed via task_id only.
op_task_attachments	OP001	P1	NO	task_id FK, uploader_id, file_name, storage_path, deleted_at	Files on a specific task. Accessed via task_id only.
op_task_templates	OP001	P2	YES	tenant_id, name_i18n JSONB, is_chain, default_priority, category	Template picker queries per tenant. Different tenants have different automation templates.
op_task_template_steps	OP001	P2	NO	template_id FK, step_order, title, due_offset_hours, depends_on_prev	Steps within one template. Always accessed via template_id.
op_task_triggers	OP001	P2	YES	tenant_id, event_type, template_id FK, condition_sql, enabled	Trigger management per tenant. Each tenant configures different business flow automations.

TABLE	MODULE	PHASE	TENANT_ID	KEY COLUMNS	REASONING
op_task_trigger_events	OP001	P2	NO	trigger_id FK, source_object_type/id, tasks_created. UNIQUE idempotency	Execution log for one trigger. Accessed via trigger_id only.
op_task_views	OP001	P2	YES	tenant_id, name, filter_config JSONB, owner_id, is_pinned, sort_order	Saved filters per user per tenant. Listed in cockpit sidebar.
op_goals	OP002	P1	YES	tenant_id, title, parent_goal_id (self-ref), entity ENUM, period_type/start/end, status, health, owner_id, vision_text, budget, deleted_at	RLS root. Goal tree queries, entity-scoped dashboard, all tenant-filtered.
op_goal_key_results	OP002	P1	YES	tenant_id, goal_id FK, metric_type ENUM, direction, baseline/target/current, weight, metric_source, unit_label, deleted_at	Dashboard summaries query KRs across goals. Metric computation filters by tenant. Nightly cron batch-reads by tenant.
op_goal_kr_snapshots	OP002	P1	NO	kr_id FK, snapshot_date, value, progress_pct, pace_delta. UNIQUE(kr_id, date)	Time-series for one KR. Sparkline reads via kr_id only. Tenant via kr_id → op_goal_key_results.
op_goal_checkins	OP002	P1	NO	goal_id FK, author_id, confidence (1-5), progress_note, blockers	Check-in history for one goal. Accessed via goal_id only.
op_goal_metrics_catalog	OP002	P2	NO	code PK, name_i18n JSONB, metric_type, unit_label, entity_scoped, query_key	System-wide reference table. Metric definitions shared across all tenants — not tenant-specific data.

6. Cross-Module Dependencies

OP001 reads from / writes to:

- bop_users — assignee_id, created_by
- listings / listing_inquiries / fin_invoices / sup_requests / crm_leads — polymorphic ref_object links + trigger sources

- `bop_comm_history` — Telegram alert logging
- `op_goal_key_results` — goal_kr_id bridge (STR-P2, added as column in OPS-P2)
- `sy_tasks.op_task_id` — SY051 implementation cockpit link (existing infrastructure)

OP002 reads from (auto-metric sources in STR-P2):

- `listings` — vans_sold, avg_days_to_sale, stock_count
- `fin_invoices` — revenue_invoiced, gross_margin_sum (margeregeling-aware)
- `listing_inquiries` / `crm_leads` — inquiries_count, inquiry_to_sale_conversion
- `reviews` — review_score_avg, nps_avg
- `op_tasks` — linked tasks per KR via goal_kr_id bridge

Coexistence with DesCampers:

- DC `tasks` table remains untouched — DC admin continues operating independently
- DC `objectives` + `key_results` tables remain untouched — DC goals screen continues as-is
- No FK links between BOP op_* tables and DC tables
- Future option: retire DC screens once BOP OP001/OP002 are fully operational

7. SY051 Program Seed — BOP-OPSCOCKPIT

Program code: `BOP-OPSCOCKPIT` — to be seeded into SY051 via SQL (following `sql_bop_v2_tfe_impl_plan.sql` pattern)

Phases: 6 (OPS-P1, STR-P1, OPS-P2, STR-P2, OPS-P3, STR-P3)

Tasks: ~48 (from the breakdown above: 13 + 9 + 9 + 7 + 5 + 4)

Deliverables: ~135 (DB_TABLE x14, SCREEN x12, API_ROUTE x4, RPC x6, PERMISSION x15, MIGRATION x15, CRON x3, TELEGRAM x8, SERVICE x2, I18N_KEY x30, plus test/verification deliverables)

Estimated total: ~25 FTE days across 6 phases

Target dates: P1 phases (OPS-P1 + STR-P1) by end of Q3 2026. P2 phases by end of Q4 2026. P3 phases on observed go-signals.

8. Recommended Build Order

Step 0: Generate `sql_bop_v2_opsc cockpit_impl_plan.sql` — register all 6 phases, ~48 tasks, ~135 deliverables in SY051 Implementation Cockpit. Tracking from day one.

Step 1 — OPS-P1: Build `op_tasks` + 4 supporting tables + RPCs + API routes + cockpit UI + task detail. SLA cron + Telegram. PBAC. Result: OP001 becomes the daily homepage.

Step 2 — STR-P1: Build `op_goals` + 3 supporting tables + API routes + goal tree + detail page. Manual KR values + check-ins. PBAC. Result: Monday check-in habit established.

Step 3 — OPS-P2: Build automation engine: templates + triggers + `fireTaskTriggers()` wired into inquiry/listing/invoice/support flows. Kanban + Calendar views. Saved filters. Result: business events auto-create tasks.

Step 4 — STR-P2: Build metrics catalog + `computeMetric()` service + nightly job + snapshots. Executive dashboard with sparklines + entity lanes. Goal-task bridge. Result: KRs update themselves from trade log + invoices.

Step 5 — P3s: Dependencies UI, Gantt, RRULE recurrence, time tracking, Telegram inline buttons, analytics, period close-out, budget linkage, forecast lines, entity scorecards. Deferred to go-signals.

Note on DesCampers: The DC admin Tasks + Goals screens (1,513 lines total) remain operational on their own tables. Once BOP OP001 + OP002 reach feature parity (end of OPS-P2 / STR-P1), the DC screens can optionally be retired or rewired to read from BOP `op_*` tables. This is a future decision — not part of this build plan.

All tables built new — no ALTER/RENAME of existing DesCampers tables

6 tables with `tenant_id` (`op_tasks`, `op_goals`, `op_goal_key_results`, `op_task_templates`, `op_task_triggers`, `op_task_views`)

9 tables without (`child/log` tables accessed via parent FK UUID + 1 system-wide catalog)

Schema: 10 ops tables · 5 strategy tables · ~15 PBAC permissions · 3 cron workers · ~8 Telegram alert types